pfi

Python for Fixed Income

COURSE 2 · BOND MATH

Treasury Curve Interpolation

WEEK 3 · DAY 1

Created by **Jeff Lenamon**

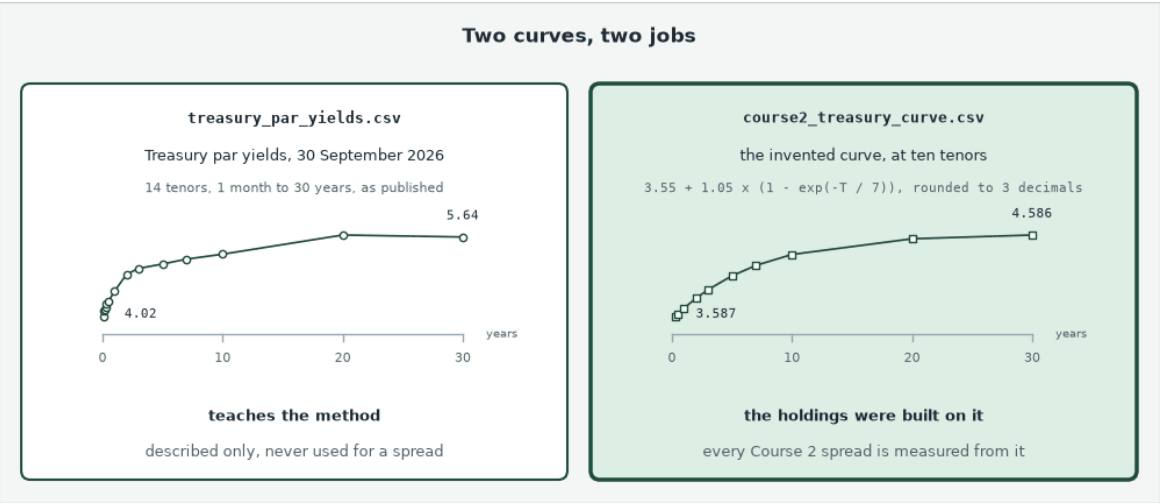
This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note, and two curves

The Desk Lead's Note

Week 3 measures every bond against Treasuries, so each bond needs a Treasury yield at its own maturity. Treasury publishes fourteen tenors. Read me the 30 September curve at 6.4 years, then put the method in "bondmath" and tie it to Excel. Tell me which curve our holdings were built on, and how much a straight line between tenors moves the number.

Every statement today describes the data; none says where yields go.



Two curves arrive in the work folder today, each with one job.

Source	U.S. Department of the Treasury, Daily Treasury Par Yield Curve Rates
Retrieved	2026-10-03, from the yearly CSV downloads
Reuse	public domain, CC0 1.0, in Treasury's data inventory

"course2_treasury_curve.csv" is invented, a smooth formula at ten tenors, and is the curve the holdings file was built on.

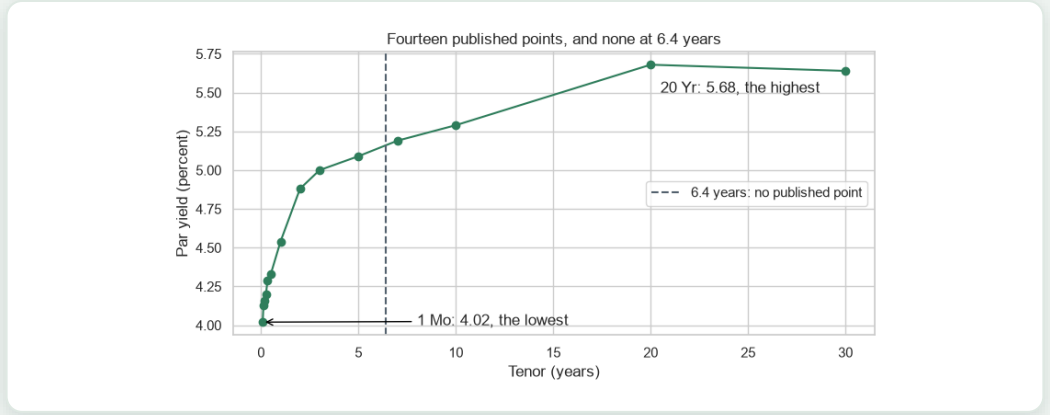
The real curve teaches the method; the invented one is the one Course 2 measures from.

11.1 The curve on 30 September 2026

	label	yield_pct	tenor_years
0	1 Mo	4.02	0.0833
1	1.5 Month	4.13	0.1250
2	2 Mo	4.16	0.1667
3	3 Mo	4.20	0.2500
4	4 Mo	4.29	0.3333
5	6 Mo	4.33	0.5000
6	1 Yr	4.54	1.0000
7	2 Yr	4.88	2.0000
8	3 Yr	5.00	3.0000
9	5 Yr	5.09	5.0000
10	7 Yr	5.19	7.0000
11	10 Yr	5.29	10.0000
12	20 Yr	5.68	20.0000
13	30 Yr	5.64	30.0000

Fourteen yields in percent, under text labels: "Mo" for most month tenors, "Month" for the 1.5-month one. A label cannot go on an axis or into a formula; a tenor in years can.

6 of the 14 tenors are under a year; the last two are 10 years apart.



The "20 Yr" sat above the "30 Yr", 5.68 against 5.64, as Course 1's Treasury case study found on its last date. The chart describes one date; it says nothing about why the curve had this shape or where it goes.

Excel side by side. Tenors in years in A2:A15 ("=1/12" for "1 Mo", and so on), yields in B2:B15.

Job	Excel	Result
Chart the curve	select A1:B15, Insert, Scatter with Straight Lines and Markers	one series, yield_pct, tenors in years on the x axis
The same range as a Line chart	select A1:B15, Insert, Line	two series, the tenor column drawn as a line, 14 points evenly spaced

Fourteen points and none at 6.4 years: the yield there comes from a line drawn between two of them.

11.2 A straight line between two tenors

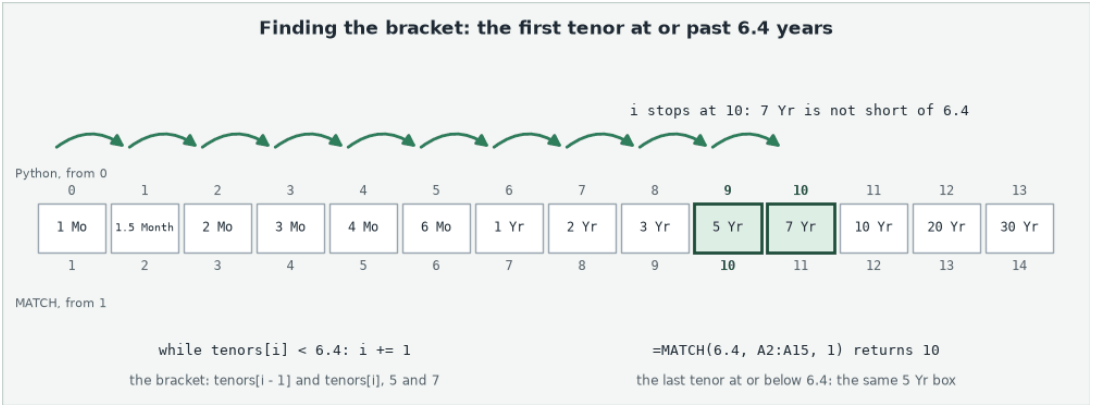
6.4 years lies between the 5-year and the 7-year tenors: they are its bracket. Linear interpolation reads the straight line between them. Its pieces: the left and right tenors, their yields, and the weight, the share of the way from the left tenor to the right one.

Excel side by side: the line, and Excel's own

Job	Excel
The line written out	=B2+(B3-B2)*(6.4-A2)/(A3-A2)
The line by Excel	=FORECAST.LINEAR(6.4,B2:B3,A2:A3)
Arguments swapped	=FORECAST.LINEAR(6.4,A2:A3,B2:B3)

"FORECAST.LINEAR" takes the yields before the tenors; swapped, it forecasts a tenor and says nothing. Yields in percent go in and come out: a straight line needs no "/100".

```
tenors = real_curve['tenor_years'].tolist()
labels = real_curve['label'].tolist()
i = 0
while tenors[i] < 6.4:
    i += 1
print(f'6.4 years lies between {labels[i - 1]} and {labels[i]}: positions {i - 1} and {i}')
6.4 years lies between 5 Yr and 7 Yr: positions 9 and 10
```



Python counts positions from 0, so "5 Yr" is position 9; Excel's "MATCH" counts from 1 and calls it 10.

Job	Excel	Result
The bracket's position	=MATCH(D1,A2:A15,1)	10, counted from 1
The bracket's tenors	=INDEX(A2:A15,MATCH(D1,A2:A15,1)), and with +1 after the MATCH	5 and 7

11.3 interpolate_yield, docstring first

```
%%writefile -a bondmath.py
def interpolate_yield(tenor_years: float, curve_tenors: list[float], curve_yields_pct: list[float]) -> float:
    if len(curve_tenors) != len(curve_yields_pct) or len(curve_tenors) == 0:
        raise ValueError("curve_tenors and curve_yields_pct must be non-empty and the same length")
    for i in range(1, len(curve_tenors)):
        if curve_tenors[i] <= curve_tenors[i - 1]:
            raise ValueError(f"curve tenors must ascend: {curve_tenors[i - 1]} then {curve_tenors[i]}")
    if tenor_years <= curve_tenors[0]:
        return curve_yields_pct[0]
    if tenor_years >= curve_tenors[-1]:
        return curve_yields_pct[-1]
    i = 1
    while curve_tenors[i] < tenor_years:
        i += 1
    left_tenor, right_tenor = curve_tenors[i - 1], curve_tenors[i]
    left_yield, right_yield = curve_yields_pct[i - 1], curve_yields_pct[i]
    slope = (right_yield - left_yield) / (right_tenor - left_tenor)
    return left_yield + slope * (tenor_years - left_tenor)
```

Three decisions go into the docstring before any code:

Units: tenors in years, yields in percent, the result in percent.

Flat ends (flat extrapolation): below the first tenor, the first yield; past the last, the last yield, so no slope is invented beyond the curve.

The check: the tenors must ascend, or the bracket search reads the wrong pair, so a jumbled curve raises "ValueError".

The checks, the flat ends, 11.2's bracket loop, then 11.2's line written as a slope: the rise per year times the years along.

```
for tenor in [10, 0.02, 40]:
    result = bondmath.interpolate_yield(tenor, real_tenors, real_yields)
    print(f'{tenor:>5} years: {result:.2f} percent')
   10 years: 5.29 percent
 0.02 years: 4.02 percent
   40 years: 5.64 percent
```

```
bondmath.interpolate_yield(4, [1, 3, 2], [3.7, 3.9, 3.8])
ValueError: curve tenors must ascend: 3 then 2
```

It refuses, and says where.

The tests, and Excel's answers

Four tests, on the invented curve of The Data, typed as "CURVE_TENORS" and "CURVE_YIELDS_PCT":

"test_on_a_tenor_returns_its_yield": each of the ten points gives back its own yield.

"test_matches_numpy_interp": nine tenors, inside and outside, agree to 1e-12 with numpy's own straight line, "np.interp" (11.4), imported inside the test because "bondmath" does not use numpy.

"test_flat_beyond_the_ends" and "test_raises_when_tenors_not_ascending": two of the docstring's decisions.

```
%%writefile -a test_bondmath.py
CURVE_TENORS = [0.25, 0.5, 1, 2, 3, 5, 7, 10, 20, 30]
CURVE_YIELDS_PCT = [3.587, 3.622, 3.69, 3.811, 3.916, 4.086, 4.214, 4.348, 4.54, 4.586]
def test_on_a_tenor_returns_its_yield():
    for tenor, yield_pct in zip(CURVE_TENORS, CURVE_YIELDS_PCT):
        assert bondmath.interpolate_yield(tenor, CURVE_TENORS, CURVE_YIELDS_PCT) == pytest.approx(yield_pct, abs=1e-12)
def test_matches_numpy_interp():
    import numpy as np  # numpy is in the course requirements; bondmath itself does not use it
    for tenor in (0.1, 0.92, 1.5, 4.0, 6.4, 8.75, 15.0, 29.9, 35.0):
        expected = float(np.interp(tenor, CURVE_TENORS, CURVE_YIELDS_PCT))
        assert bondmath.interpolate_yield(tenor, CURVE_TENORS, CURVE_YIELDS_PCT) == pytest.approx(expected, abs=1e-12)
def test_flat_beyond_the_ends():
    assert bondmath.interpolate_yield(0.1, CURVE_TENORS, CURVE_YIELDS_PCT) == 3.587
    assert bondmath.interpolate_yield(40, CURVE_TENORS, CURVE_YIELDS_PCT) == 4.586
def test_raises_when_tenors_not_ascending():
    with pytest.raises(ValueError):
        bondmath.interpolate_yield(4, [1, 3, 2], [3.7, 3.9, 3.8])
```

```
...   [100%]
32 passed in 0.50s
```

Excel reads a whole curve with one formula: "MATCH" finds the row, and "INDEX" hands two points to "FORECAST.LINEAR". The curve in A2:B15, the tenor to read in D1.

```
Any tenor inside
=FORECAST.LINEAR(D1,INDEX(B2:B15,MATCH(D1,A2:A15,1)):INDEX(B2:B15,
MATCH(D1,A2:A15,1)+1),INDEX(A2:A15,MATCH(D1,A2:A15,
1)):INDEX(A2:A15,MATCH(D1,A2:A15,1)+1))
```

Below the first tenor	flat: 4.02 at 0.02	#N/A
At or past the last	5.64 at 30, and flat at 35	#REF!

Use the formula strictly inside the curve; the flat ends are the function's job.

```
13 tenors; largest gap from Excel: 8.9e-16 percent
```

Excel's answers inside the curve, flat outside it, a refusal for a jumbled curve: 32 tests pass.

11.4 The same with numpy.interp

numpy has the straight line built in: "np.interp(x, tenors, yields)", for one tenor or a list of them.

```
eight_tenors = [0.02, 0.5, 2.5, 6.4, 8.5, 15, 25, 40]
comparison = pd.DataFrame({'tenor_years': eight_tenors})
comparison['ours'] = [bondmath.interpolate_yield(t, real_tenors, real_yields)
                      for t in eight_tenors]
comparison['numpy'] = np.interp(eight_tenors, real_tenors, real_yields)
comparison['difference'] = comparison['ours'] - comparison['numpy']
comparison
```

Every tenor agrees, both flat ends included.

tenor_years	ours	numpy	difference
0.02	4.020	4.020	0.0
0.50	4.330	4.330	0.0
2.50	4.940	4.940	0.0
8.50	5.240	5.240	0.0
15.00	5.485	5.485	0.0
25.00	5.660	5.660	0.0
40.00	5.640	5.640	0.0

11.5 The curve the holdings were built on

The holdings file's yields were built on the invented curve, not on Treasury's; the repo's "DATA.md" gives its formula:

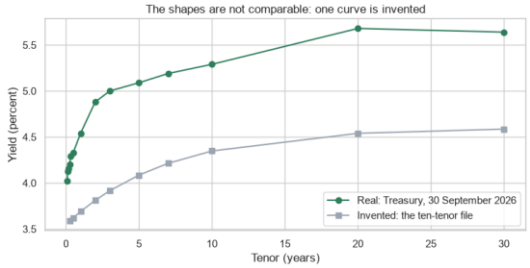
yield in percent = 3.55 + 1.05 x (1 - exp(-T / 7)), with T the years to maturity.

It starts at 3.55, levels off toward 3.55 + 1.05 = 4.60, and 7 sets how fast; "exp" is Course 1's "np.exp". At T = 7, exp(-1) is 0.3679, so the yield is 3.55 + 1.05 x 0.6321 = 4.2137 percent, 4.214 to 3 decimals. The program that wrote the file added each bond's spread to it.

```
invented = pd.read_csv('course2_treasury_curve.csv')
invented_tenors = invented['tenor_years'].tolist()
invented_yields = invented['yield_pct'].tolist()
formula_pct = np.round(3.55 + 1.05 * (1 - np.exp(-invented['tenor_years'] / 7)), 3)
assert (formula_pct == invented['yield_pct']).all()
invented
```

Each point is the formula rounded to 3 decimals; notebook 12 meets that rounding again. The table starts at 0.25 years because Course 1's October book holds a bond with 0.92 years left; the holdings run from 1.01 to 29.90 years, all inside it. Excel side by side. With the ten points in A2:B11 and the formula's ranges ending at row 11, Excel returns 4.1756 at 6.4 years, "#N/A" at 0.1 and "#REF!" at 30, as on the real curve.

tenor_years	yield_pct
0.25	3.587
0.50	3.622
1.00	3.690
2.00	3.811
3.00	3.916
5.00	4.086
7.00	4.214
10.00	4.348
20.00	4.540
30.00	4.586



The shapes are not comparable, because one curve is invented: it was written to give synthetic bonds believable yields, never fitted to Treasury's figures.

The tests' ten points, and what Git leaves out

```
assert CURVE_TENORS == invented_tenors
assert CURVE_YIELDS_PCT == invented_yields
print(len(CURVE_TENORS), 'points: the tests type exactly the file')
10 points: the tests type exactly the file
```

```
!git -C "{work}" status --short --ignored
!git -C "{work}" diff --stat
 M bondmath.py
 M test_bondmath.py
!! __pycache__/
!! bondmath_excel_reference.csv
!! bondmath_excel_reference_tvm.csv
!! course2_treasury_curve.csv
!! treasury_par_yields.csv
 bondmath.py      | 30 ++++++++++++++++++++++++++++++
 test_bondmath.py | 29 ++++++++++++++++++++++++++++++
 2 files changed, 59 insertions(+)
```

"!!" marks what Git leaves out: "__pycache__" by notebook 06's ".gitignore", the data files because "start_day" lists them in Git's own ignore list. The curve file is data: it comes from its source, the repo's data folder, not from the module's record.

```
!git -C "{work}" add bondmath.py test_bondmath.py
subject = 'Add interpolate_yield: linear between tenors, flat beyond the ends'
body = 'Week 3 reads a curve yield at each maturity; the tests hold it to numpy.interp.'
!git -C "{work}" commit -q -m "{subject}" -m "{body}"
!git -C "{work}" log --oneline -1
fa6939c Add interpolate_yield: linear between tenors, flat beyond the ends
```

Every Course 2 spread is measured from this curve: the tests type its ten points, and the history holds the module and its tests, never the data.

Bring it home, and what you can now do

You Can Now

Turn Treasury's tenor labels into years and chart a curve at true distances.

Read a curve between two tenors by hand, as a weighted average of the neighbours.

Find the bracket with a loop, and with Excel's "MATCH" and "INDEX".

Read any tenor with "interpolate_yield", tied to Excel's "FORECAST.LINEAR" inside the curve, flat beyond it, refusing a jumbled curve.

Put a number on what a straight line between tenors misses.

See what Git leaves out with "git status --short --ignored", and commit the module and its tests without the data.

Copy the work folder's two files into your "my_bondmath" folder; the copy replaces yours, so anything of your own must be in the work folder first. In PowerShell in VS Code, with the course's ".venv" active and the folder name the setup printed:

```
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$env:TEMP\pfi_course2_11_k4w7q2xr\bondmath.py", "$env:TEMP\pfi_course2_11_k4w7q2xr\test_bondmath.py" .
python -m pytest -q
git diff --stat
git commit -am "Add interpolate_yield: linear between tenors, flat beyond the ends"
```

pytest shows 32 passed, more with your exercise tests. In Colab, download the two files from the work folder under "/tmp".

Predict first: 6.4 years, and numpy's straight line

Predict 1

On 30 September 2026 the 5-year yielded 5.09 percent and the 7-year 5.19. Read a straight line between them at 6.4 years. Write down a number.

```
left_tenor, right_tenor = 5, 7
left_yield, right_yield = curve_row['5 Yr'], curve_row['7 Yr']
weight = (6.4 - left_tenor) / (right_tenor - left_tenor)
yield_at_tenor = left_yield + weight * (right_yield - left_yield)
print(f'weight {weight:.1f}: {yield_at_tenor:.4f} percent at 6.4 years')
```

Predict 2

11.3's jumbled curve stopped "interpolate_yield". Hand it to numpy and ask for 4 years: "np.interp(4, [1, 3, 2], [3.7, 3.9, 3.8])".

A) it raises a "ValueError", as "interpolate_yield" did

B) it sorts the tenors first and returns 3.9

C) it keeps the list as typed and returns its last yield

```
print(np.interp(4, [1, 3, 2], [3.7, 3.9, 3.8]))
```

Predict first: where the straight line misses most

Between its ten points the invented curve bends; "interpolate_yield" draws straight lines.

Predict 3

Read the invented curve every 0.01 years, from the formula and from the straight lines through its ten points. Where are the two furthest apart?

A) under 1 year, where the curve rises fastest

B) between 10 and 20 years

C) between 20 and 30 years, the last stretch

```
grid = np.linspace(0.25, 30, 2976)
from_formula = 3.55 + 1.05 * (1 - np.exp(-grid / 7))
from_lines = np.array([bondmath.interpolate_yield(t, invented_tenors, invented_yields)
                       for t in grid])
gap_bp = (from_formula - from_lines) * 100
print(f'largest gap: {gap_bp.max():.2f} bp, at {grid[gap_bp.argmax()]:.2f} years')
print(f'average gap: {gap_bp.mean():.2f} bp')
```

The note, answered

The note asked	Answer
The yield at 6.4 years	5.16 percent; Excel's FORECAST.LINEAR draws the same line, 5.160000000000001
The method in bondmath, tied to Excel	interpolate_yield, flat beyond the ends, refusing a jumbled curve; 13 Excel values within 8.9e-16; 32 passed
The curve the holdings were built on	the invented ten-tenor curve, its formula rounded to 3 decimals
What a straight line moves	up to 3.33 bp, at 14.39 years; 1.01 bp on average

Be proud: you can read a curve at any maturity and say what the reading leaves out.

Next: Spread Over the Curve (notebook 12) rebuilds the holdings file's "oas" column on this curve.